

A gamified introduction to Python Programming

Lecture 2

Basic Concepts of Programming

Matthieu DE MARI – Singapore University of Technology and Design



SINGAPORE UNIVERSITY OF
TECHNOLOGY AND DESIGN



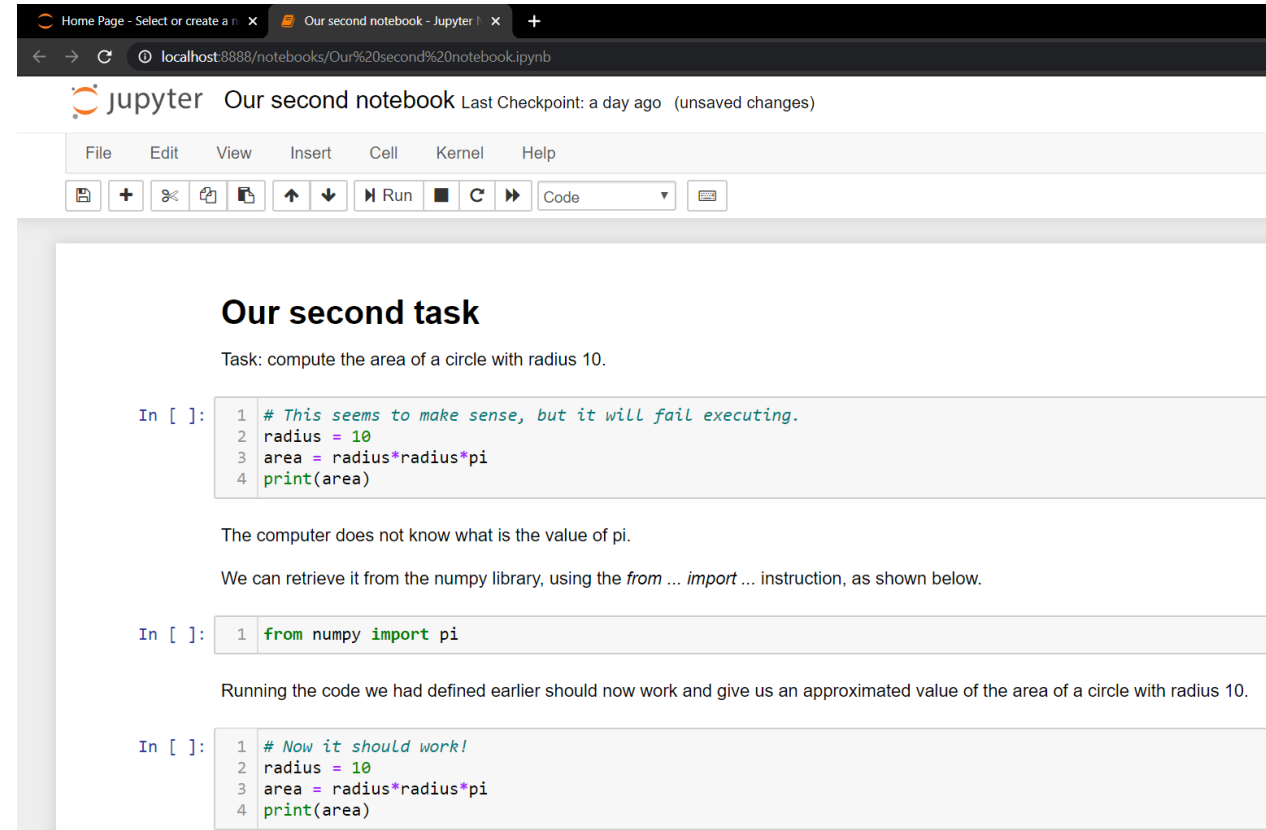
Outline (Chapter 2)

- **Everything you need to know about variables:** Naming conventions, types, IDs, assignments, types conversions.
- **Using basic math operators in Python on mathematical variables:** `+`, `-`, `*`, `/`, `//`, `%`, `**`.
- **Combining operators and assignments.**
- **Commenting:** in-line, block, docstring and header comments.
- **Printing and getting:** displaying with `print()`, getting with `input()`.
- And more things!

Recall: our second task in Lesson 1

In this task, we have intuitively used a few concepts, such as:

1. Assigning values to memory (variables)
2. Using basic math operations
3. Commenting your code
4. Printing
5. Executing a coding script in console or an IDE
6. ~~(Importing stuff, for later!)~~



Home Page - Select or create a notebook | Our second notebook - Jupyter | +

localhost:8888/notebooks/Our%20second%20notebook.ipynb

jupyter Our second notebook Last Checkpoint: a day ago (unsaved changes)

File Edit View Insert Cell Kernel Help

+ %< >% + Run [] Code []

Our second task

Task: compute the area of a circle with radius 10.

```
In [ ]: 1 # This seems to make sense, but it will fail executing.
        2 radius = 10
        3 area = radius*radius*pi
        4 print(area)
```

The computer does not know what is the value of pi.

We can retrieve it from the numpy library, using the `from ... import ...` instruction, as shown below.

```
In [ ]: 1 from numpy import pi
```

Running the code we had defined earlier should now work and give us an approximated value of the area of a circle with radius 10.

```
In [ ]: 1 # Now it should work!
        2 radius = 10
        3 area = radius*radius*pi
        4 print(area)
```

→ In the next couple of slides, we will go deeper into these concepts.

About variables names

Definition (**Naming Convention** for variables in programming):

- **Variables names** can include any combination of **letters** (both lowercase and uppercase), **digits** and **underscore symbols** (_).
- Variables names **should NOT start with a digit**.
- Some **keywords** (e.g. import) are protected and should not be used.

```
1 # This is okay as a variable name
2 a_1st_VaRiAbLe = 1
```

```
1 # Variables cannot start with a digit
2 2nd_variable = 2
```

```
File "<ipython-input-3-f01f45c0ae38>", line 2
    2nd_variable = 2
    ^
```

SyntaxError: invalid syntax

```
1 # But they can start with an underscore (_)
2 _third_variable = 3
```

```
1 # In fact, this is also a valide name!
2 # (But not explicit at all!)
3 _ = 4
4 print(_)
```

4

```
1 # Variables cannot use special symbols,
2 # except for the underscore symbol (_)
3 v@r!aBl€ = 5
```

```
File "<ipython-input-8-f47b51204aa7>", line 2
    v@r!aBl€ = 5
    ^
```

SyntaxError: invalid syntax

```
1 # Cannot use keywords as variable names
2 import = 6
```

```
File "<ipython-input-20-4526a1d99ee1>", line 2
    import = 6
    ^
```

SyntaxError: invalid syntax

Matt's Great Advice

Matt's Great Advice: Try making your variables names explicit.

It is a good idea to make your **variables names explicit** (but not too much!) rather than using meaningless ones that leave your reader guessing what they stand for (x, y, a, b1, c2, etc.).

You can **use underscores** if you find it helpful (e.g. `current_year = 2020`).

Your future self, and colleagues, will thank you, when they work on your code later on.



```
# A variable name that is not explicit (what does x stand for?)  
x = 10
```

```
# A variable name that is more explicit  
radius = 10
```

```
# A variable name that is, maybe, a bit too explicit?  
the_wonderful_birth_year_of_your_lovely_instructor = 1989
```



What happens if you use the command "print = 5" in Python?

① Start presenting to display the poll results on this slide.

Multiple assignments

Python also allows **multiple assignments** using

1. **Successive multiple equal signs:**
the value on the far right is assigned to all variables names identically.
2. **Or by having several variables names and values separated by commas on both sides:**
the values are respectively assigned to the variables names, in a one for one manner.

```
1 # A multiple assignment
2 var1 = var2 = var3 = 10
3 print(var1)
4 print(var2)
5 print(var3)
```

```
10
10
10
```

```
1 # Another multiple assignment
2 var4, var5 = 8, "Some text"
3 print(var4)
4 print(var5)
```

```
8
Some text
```

Variable types

Definition (**variable type/class**):

Everything in your computer is stored in memory as 0s and 1s.

A **variable type/class** defines how the sequence of 0s and 1s should be interpreted by the computer.

Let us start with three basic types:

- **int**: an integer number.
- **float**: a floating point number, for decimal numbers.
- **str**: stands for string, a bunch of text.

```
1 radius = 10
2 pi = 3.14
3 message = "Hello World!"
```

```
1 print(type(radius))
```

```
<class 'int'>
```

```
1 print(type(pi))
```

```
<class 'float'>
```

```
1 print(type(message))
```

```
<class 'str'>
```


Variable IDs

Definition (**variable ID**):

Everything in your computer is stored in memory as 0s and 1s.

A **variable ID** returns a number, corresponding to a memory address where the variable is being stored in memory.

The storing process for variables is automatically handled by your computer. More on this later, so let us keep it in mind.

```
1 radius = 10
2 pi = 3.14
3 message = "Hello World!"
```

```
1 print(id(radius))
```

```
140721979856832
```

```
1 print(id(pi))
```

```
2367280141872
```

```
1 print(id(message))
```

```
2367280157744
```

Type conversion

Int/Float → String: You can convert any int/float to a string using the **str()** function.

- **Note:** Both the int/float and the str variables might print the same thing on screen, but they are not equivalent when it comes to what is stored in memory and how these variables can be used!

For your computer, these two variables are very different things!

```
1 a_number_as_int = 1024
2 same_number_as_string = str(a_number_as_int)
3 print(a_number_as_int)
4 print(type(a_number_as_int))
5 print(same_number_as_string)
6 print(type(same_number_as_string))
```

```
1024
<class 'int'>
1024
<class 'str'>
```

```
1 a_number_as_float = 1.5
2 same_number_as_string = str(a_number_as_float)
3 print(a_number_as_float)
4 print(type(a_number_as_float))
5 print(same_number_as_string)
6 print(type(same_number_as_string))
```

```
1.5
<class 'float'>
1.5
<class 'str'>
```

Type conversion

String → Int/Float: You can convert a string of digits to an int/float using **int()** or **float()**.

- A string containing only digits can be converted to int or float.
- A string of digits may contain a single decimal point. In that case, it can be converted to a float, but not to an int.
- Everything else fails (two decimal points, special characters, etc.).

```
1 number_as_string = "1024"
2 same_number_as_int = int(number_as_string)
3 same_number_as_float = float(number_as_string)
4 print(number_as_string)
5 print(type(number_as_string))
6 print(same_number_as_int)
7 print(type(same_number_as_int))
8 print(same_number_as_float)
9 print(type(same_number_as_float))
```

```
1024
<class 'str'>
1024
<class 'int'>
1024.0
<class 'float'>
```

Type conversion

String → Int/Float: You can convert a string of digits to an int/float using **int()** or **float()**.

- A string containing only digits can be converted to int or float.
- A string of digits may contain a single decimal point. In that case, it can be converted to a float, but not to an int.
- Everything else fails (two decimal points, special characters, etc.).

```
# str to float conversion (decimal point in string)
number_as_string = "1.5"
same_number_as_float = float(number_as_string)
print(number_as_string)
print(type(number_as_string))
print(same_number_as_float)
print(type(same_number_as_float))
```

```
1.5
<class 'str'>
1.5
<class 'float'>
```

```
# str to int conversion (decimal point in string)
same_number_as_int = int(number_as_string)
```

```
-----
ValueError                                Traceback
<ipython-input-27-dc9c39dc6e81> in <module>
----> 1 same_number_as_int = int(number_as_string)

ValueError: invalid literal for int() with base 10
```

Type conversion

Int $\leftrightarrow$ Float: You can convert a float to an int and vice versa.

- Converting from int to float adds a single 0 decimal.
- Also, converting from float to int, will **drop all decimals** (i.e. a truncation of all decimals, which is NOT the same as rounding!).

Feel free to experiment!

```
1 an_int_number = 12
2 int_to_float_number = float(an_int_number)
3 print(int_to_float_number)
```

12.0

```
1 a_float_number = 1.8
2 float_to_int_number = int(a_float_number)
3 print(float_to_int_number)
```

1

Python basic operators

By default, Python comes with a few **basic math operators, for number (int/float) variables.**

- **+**: addition
- **-**: subtraction
- *****: multiplication
- **/**: division
- **//**: integer division
- **%**: remaining of the integer division (a.k.a. modulus)
- ******: exponentiation

```
1 # Two numbers
2 a, b = 5, 2
3 print(a)
4 print(b)
```

5
2

```
1 # Addition
2 print(a+b)
```

7

```
1 # Subtraction
2 print(a-b)
```

3

```
1 # Multiplication
2 print(a*b)
```

10

```
1 # Division
2 print(a/b)
```

2.5

```
1 # Floor division
2 print(a//b)
```

2

```
1 # Modulus
2 print(a%b)
```

1

```
1 # Exponentiation
2 print(a**b)
```

25

Python basic operators

By default, Python comes with a few **basic math operators**, for **number (int/float) variables**.

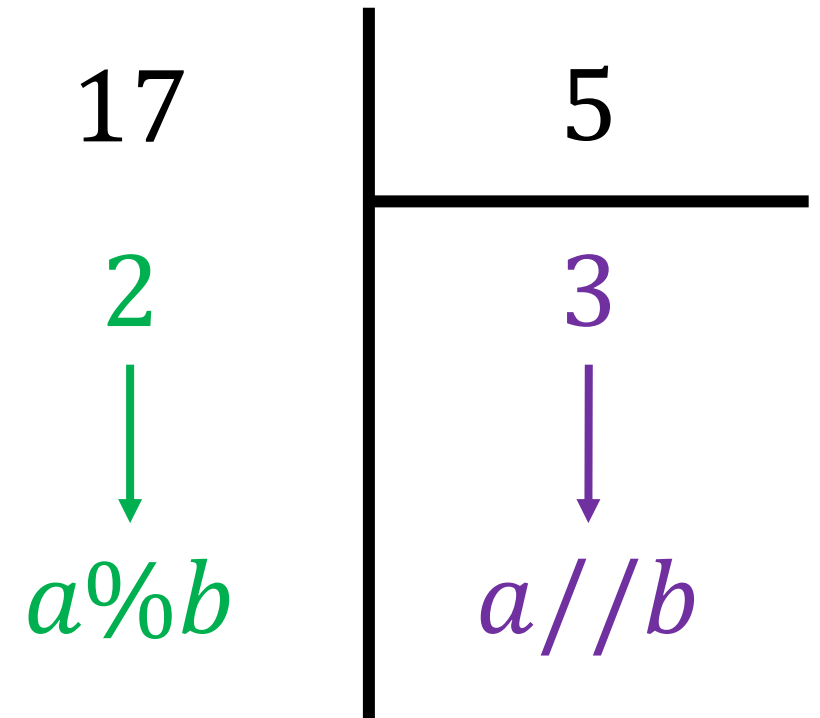
- **+**: addition
- **-**: subtraction
- *****: multiplication
- **/**: division
- **//**: integer division
- **%**: remaining of the integer division (a.k.a. modulus)
- ******: exponentiation

```
1 # Floor division
2 print(a//b)
```

2

```
1 # Modulus
2 print(a%b)
```

1



Python basic operators

By default, Python comes with a few **basic math operators**, for number (int/float) variables.

- **+**: addition
- **-**: subtraction
- *****: multiplication
- **/**: division
- **//**: integer division
- **%**: remaining of the integer division (a.k.a. modulus)
- ******: exponentiation

```
a = 2
b = "1024"
print(a+b)
```

```
-----
TypeError                                Traceback (most recent call last)
Input In [14], in <cell line: 3>()
      1 a = 2
      2 b = "1024"
----> 3 print(a+b)

TypeError: unsupported operand type(s) for +: 'int' and 'str'
```


Combining operators and assignments

You can mix both mathematical and memory assignments operator.

- For instance, the **operation +=** sums values on the right- and left-hand sides and assigns the result to the variable on the left-hand side.
- Similarly, we have the following operators: -=, *=, /=, //=, %=, **=.
- Can you guess what they do?

```
1 # Addition
2 a, b = 5, 2
3 print(a)
4 a += b
5 print(a)
```

5
7

```
1 # Multiplication
2 a, b = 5, 2
3 print(a)
4 a *= b
5 print(a)
```

5
10

Operation precedence

Precedence order in Python: Operations are always computed **from left to right**.

However, **operation precedence rules** apply:

- **Exponentiation** has higher precedence than **multiplications/divisions/modulus**, which have higher precedence than **additions/subtractions**.
- As in mathematics, **parentheses** can be used to “force” precedence, as they will always have the highest precedence.

```
1 # Operation precedence
2 a, b, c = 4, 3, 2
3 result = a+b*c
4 print(result)
```

10

```
1 # Operation precedence
2 a, b, c = 4, 3, 2
3 result = (a+b)*c
4 print(result)
```

14

```
1 # Operation precedence
2 a, b, c = 4, 3, 2
3 result = a*b**c
4 print(result)
```

36

Activity 1: Compute the roots of a quadratic equation, with strictly positive determinant

Consider the quadratic equation

$$ax^2 + bx + c = 0$$

With $a, b, c \in \mathbb{R}$, such that the determinant $\Delta > 0$

$$\Delta = b^2 - 4ac$$

As seen in math classes, the two roots of this equation (x_1, x_2) are given by

$$x_1 = \frac{-b + \sqrt{\Delta}}{2a} \quad \text{and} \quad x_2 = \frac{-b - \sqrt{\Delta}}{2a}$$

- **Task:** Consider a quadratic equation with the following values: $a = 2$, $b = -2$ and $c = -24$.

Write a Python program that

1. computes the value of the **determinant** Δ and **prints** it on screen (to check it is strictly positive),
2. And, later on, computes the values of the **roots** x_1 and x_2 and **prints** them on screen.

Activity 1: Compute the roots of a quadratic equation, with strictly positive determinant

- For this activity, you may now open and use the notebook

Activity 1 - computing the roots of a quadratic equation.ipynb

- Remember to start a notebook environment using the command below, in a console, with location appropriately selected

py -m notebook

- Also, remember, the **square root of x** is equivalent to **x being exponentiated to the power 0.5 (or import numpy sqrt function!)**

$$\sqrt{x} = x^{0.5} = x ** (0.5)$$

Activity 1: Compute the roots of a quadratic equation, with strictly positive determinant

Consider the quadratic equation

$$ax^2 + bx + c = 0$$

With $a, b, c \in \mathbb{R}$, such that the determinant $\Delta > 0$

$$\Delta = b^2 - 4ac$$

As seen in math classes, the two roots of this equation (x_1, x_2) are given by

$$x_1 = \frac{-b + \sqrt{\Delta}}{2a} \quad \text{and} \quad x_2 = \frac{-b - \sqrt{\Delta}}{2a}$$

- **Task:** Consider a quadratic equation with the following values: $a = 2$, $b = -2$ and $c = -24$.

Write a Python program that

1. computes the value of the **determinant** Δ and **prints** it on screen (to check it is strictly positive),
2. And, later on, computes the values of the **roots** x_1 and x_2 and **prints** them on screen.

→ Expected answers: $\Delta = 196$, $x_1 = 4$, $x_2 = -3$.

Activity 1: Solution

```
1 # 1. Define the coefficients (a,b,c) of the quadratic equation
2 a = 2
3 b = -2
4 c = -24
5
6 # 2. Compute the discriminant delta
7 delta = b**2 - 4*a*c
8
9 # 3. Print delta to check it is strictly positive
10 print(delta)
11
12 # 4. Compute the two roots (x1, x2)
13 x1 = (-b + delta**0.5)/(2*a)
14 x2 = (-b - delta**0.5)/(2*a)
15
16 # 5. Print the roots
17 print(x1)
18 print(x2)
```

```
196
4.0
-3.0
```

The solved notebook is in the
Activity Solutions subfolder!

Single line comments

Definition (single line comments):

Single line comments help the reader understand what is happening in your code.

On heavy programming projects, with several programmers, these are **simply essential**.

Single line comments are also great if you want to use **separators** in your code.

Personally, I use **# -----**.

```
# -----  
# 1. Define the coefficients (a,b,c) of  
# the quadratic equation  
  
a = 2  
b = -2  
c = -24  
  
# -----  
# 2. Compute the discriminant delta  
  
delta = b**2 - 4*a*c  
  
# -----  
# 3. Print delta to check it is strictly positive  
  
print(delta)  
  
# -----  
# 4. Compute the two roots (x1, x2)  
  
x1 = (b + delta**0.5)/(2*a)  
x2 = (b - delta**0.5)/(2*a)  
  
# -----  
# 5. Print the roots  
  
print(x1)  
print(x2)
```

Block comments

Definition (**block comments**):

Block comments are comments starting and ending with **triple quotation marks** (""" or ''').

These are great for **headers** in your files. Typically used, for instance, to inform the reader about the author, give a brief summary of what the code does, etc.

```
1  """
2  Author: Matthieu DE MARI
3
4  Description: computes and prints the determinant and the roots,
5  of a given quadratic equation  $ax^2 + bx + c = 0$ ,
6  with strictly positive determinant.
7
8  Inputs: Requires the user to specify values of the parameters
9  a, b, and c. These should be numbers, ideally floats.
10
11 Outputs: this script calculates and prints the values of the
12 determinant and both roots.
13
14 Important note: if the determinant is not strictly positive,
15 then the roots will be incorrect.
16 """
17
18
```


Block comments

Definition (**block comments**):

Block comments are comments starting and ending with **triple quotation marks** (""" or ""').

These are great for **headers** in your files. Typically used, for instance, to inform the reader about the author, give a brief summary of what the code does, etc.

```
1  """
2  Author: Matthieu DE MARI
3
4  Description: computes and prints the determinant and the roots,
5  of a given quadratic equation  $ax^2 + bx + c = 0$ ,
6  with strictly positive determinant.
7
8  Inputs: Requires the user to specify values of the parameters
9  a, b, and c. These should be numbers, ideally floats.
10
11 Outputs: this script calculates and prints the values of the
12 determinant and both roots.
13
14 Important note: if the determinant is not strictly positive,
15 then the roots will be incorrect.
16 """
17
18
19 # -----
20 # 1. Define the coefficients (a,b,c) of
21 # the quadratic equation
22
23 a = 2
24 b = -2
25 c = -24
26
27 # -----
28 # 2. Compute the discriminant delta
29
30 delta = b**2 - 4*a*c
31
32 # -----
33 # 3. Print delta to check it is strictly positive
34
35 print(delta)
36
37 # -----
38 # 4. Compute the two roots (x1, x2)
39
40 x1 = (b + delta**0.5)/(2*a)
41 x2 = (b - delta**0.5)/(2*a)
42
43 # -----
44 # 5. Print the roots
45
46 print(x1)
47 print(x2)
```

Matt's Great advice

Matt's Great Advice: Comment your own code... Seriously... Do it.

It is always a good idea to **comment** your code, to let the reader know what your script does.

Either use **#** for **single line comments** or triple quotations (""") for **block comments**.

You can even use **separators (# -----)**, for readability, if needed.

Again, your future self and colleagues will be grateful, when they read code you have written.



About the `print()` function

You can print multiple variables at once, using a single `print()`, by simply separating the different variables with commas.

- You can even mix several types of variables inside of a single print, using commas as a separator.
- The `print()` function always tries its best to show you the values of what you are requesting!

```
1 # Some values
2 a = 10
3 b = 8
4 print(a, b)
```

10 8

```
1 # Some more values
2 a = 10
3 b = 8
4 c = 4
5 message = "The values of (a,b,c) are:"
6 print(message, a, b, c)
```

The values of (a,b,c) are: 10 8 4

About the `print()`, and the `format()` function

Definition (the `format()` method): The `format()` method can be used to insert the value of a variable in another string variable.

- It requires **some placeholders `{}`** in the string, and **some variables** passed to the `format()` method. Placeholders are simply replaced with the values of the variables passed to the `format()` method.
- **Note:** `format()` is a special kind of function called a method. Notice how it is applied to a string variable using the **dot operator**.

```
1 a = 10
2 b = 8
3 c = 4
4 message = "The values of (a,b,c) are: {}, {}, and {}".format(a, b, c)
5 print(message)
```



The values of (a,b,c) are: 10, 8, and 4

Getting values from a user with `input()`

Definition (the `input()` function):

You can also have Python ask the user for values explicitly. This is done with the `input()` function.

- Notice how it first **displays** the string variable `message` and a submission box,
- And later **assigns** the value typed by the user in the variable on the left-hand side of the equal sign used for `input()`.

```
1 message = "Please enter a number and press enter: "  
2 user_value = input(message)  
3 second_message = "The number you entered is {}".format(user_value)  
4 print(second_message)
```

Please enter a number and press enter:

```
1 message = "Please enter a number and press enter: "  
2 user_value = input(message)  
3 second_message = "The number you entered is {}".format(user_value)  
4 print(second_message)
```

Please enter a number and press enter: 5
The number you entered is 5.

Getting values from a user with `input()`

Important: Be careful when getting values with `input()`!

- In this program, I asked the user to enter a number,
- Doubled the value of said number,
- And later displayed both the original number and its doubled value.

Problem: Something strange occurs... But why?

```
1 message = "Please enter a number and press enter: "  
2 user_value = input(message)  
3 doubled_value = 2*user_value  
4 second_message = "The number you entered is {}. Its doubled value is {}".format(user_value, doubled_value)  
5 print(second_message)
```

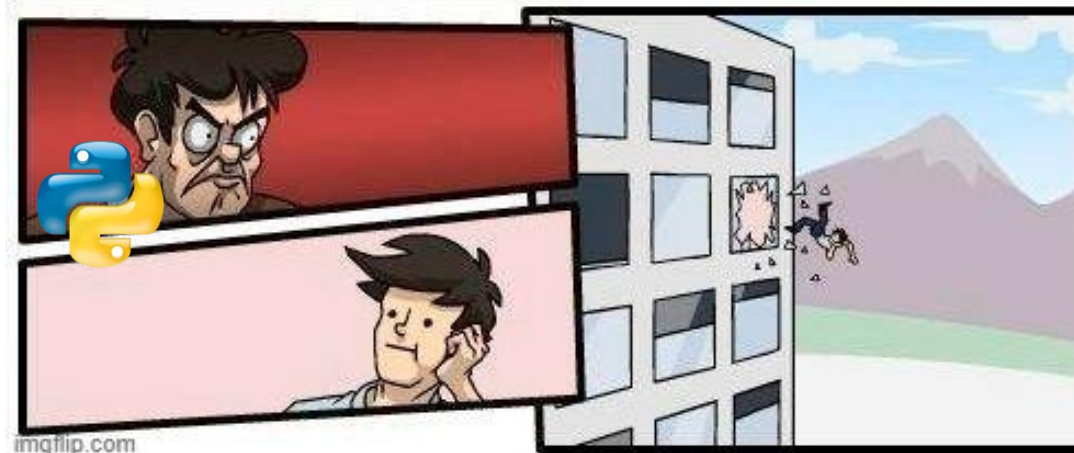
```
Please enter a number and press enter: 5  
The number you entered is 5. Its doubled value is 55.
```


Please enter a number and press enter: 5
The number you entered is 5. Its doubled value is 55.

Its doubled value is 55.

55.

Has Python gone mad?



Getting values from a user with `input()`

Important note: The values retrieved from users with `input()` will always be typed as **string variables**!

Reminder: we do not do math with strings!

By default, Python comes with a few **basic math operators, for number (int/float) variables.**

- **+**: addition
- **-**: subtraction
- *****: multiplication
- **/**: division
- **//**: integer division
- **%**: remaining of the integer division (a.k.a. modulus)
- ******: exponentiation



Watch out for the types of variables!

```
1 a = 2
2 b = "Hello World!"
3 print(a+b)
```

```
-----
TypeError                                 Traceback (most recent call last)
<ipython-input-14-4eaf3ef22f17> in <module>
      1 a = 2
      2 b = "Hello World!"
----> 3 print(a+b)
```

TypeError: unsupported operand type(s) for +: 'int' and 'str'

Getting values from a user with `input()`

Important note: The values retrieved from users with `input()` will always be typed as **string variables**!

Fun fact: It turns out that multiplying a string “5” with an integer number n simply **repeats** the string n times!

- Hence, “5”*2 is indeed “55”.

→ **Solution:** We need to convert our string variable to a float number variable before doing any math operations!

How to spam your enemies

```
# A great way to spam
text = "spam"
n = 10000
var = n*text
print(var)
```

[illegible]

Fixing our faulty code

```
1 print(type(user_value))
```

```
<class 'str'>
```

```
1 a = '5'
2 b = 2
3 c = a*b
4 print(c)
5 print(type(c))
```

```
55
<class 'str'>
```

```
1 message = "Please enter a number and press enter: "
2 user_value = input(message)
3 user_value_as_float = float(user_value)
4 doubled_value = 2*user_value_as_float
5 second_message = "The number you entered is {}. Its doubled value is {}.".format(user_value_as_float, doubled_value)
6 print(second_message)
```

```
Please enter a number and press enter: 5
The number you entered is 5.0. Its doubled value is 10.0.
```

Matt's Great advice

Matt's Great Advice: Watch out for variables types with basic math operators!

The behavior of a basic math **operator** (+, -, *, etc.) might **change**, depending on the **types of the variables** involved in the calculation.

This is called **operator overloading** and is a perfectly normal feature of Python programming.

For instance, **multiplying a string and a number variables is not the same as multiplying two number variables!**



Activity 2: Ask users about personal details

Task: Write a Python script that explicitly asks the user for its name and its birth year using text prompts. Later on, the scripts should display a message, reading

Your name is _____, and your age is either _____ or _____.

- You can use the second notebook

Activity 2 - Asking users for personal details.ipynb

(If you finish early, have a look at the extra practice/challenges folder!)

Activity 2: Ask users about personal details

```
# 1. Ask user for his/her name.
user_name = input("What is your name? ")

# 2. Ask user for his/her age.
user_birth_year = int(input("What is your birth year? "))

# 3. Compute the two possible ages the person could be.
# (Change current year value if needed!)
current_year = 2024
age1 = current_year - user_birth_year
age2 = age1 - 1

# 4. Create and format message to be displayed.
message = "Your name is {} and your age is either {} or {}!".format(user_name, age1, age2)
mean_message = "Man, you're really old."

# 5. Print the message!
print(message)
print(mean_message)
```

```
What is your name? Matt
What is your birth year? 1989
Your name is Matt and your age is either 35 or 34!
Man, you're really old.
```




Conclusion (Chapter 2)

- **Everything you need to know about variables:** Naming conventions, types, IDs, assignments, types conversions.
- **Using basic math operators in Python on mathematical variables:** `+`, `-`, `*`, `/`, `//`, `%`, `**`.
- **Combining operators and assignments.**
- **Commenting:** in-line, block, docstring and header comments.
- **Printing and getting:** displaying with `print()`, getting with `input()`.
- And more things!